Heaven's Light is Our Guide



Department of Electrical & Electronic Engineering

Rajshahi University of Engineering & Technology

Lab Report

Course No. : **EEE 3100**

Course Title : **Electronics Shop Practice**

Experiment No. : 07

Experiment Name : Control and Analyze the Operation of DC Motors Using Arduino and L293D Motor Driver IC

Submitted By:

Name : Md. Raiyan Mahmud

Roll : 2201042

Section : A

Submitted To:

Md. Khayrul Islam

Lecturer

Dept. of EEE, RUET

Date of Experiment : 22 April, 2026

Date of Submission : 06 May, 2026

Obtained Marks	Signature	Comments

Experiment No.: 07

Name of the Experiment: Control and Analyze the Operation of DC Motors Using Arduino and L293D Motor Driver IC

Objectives:

1. To understand the working principle of the L293D H-Bridge motor driver IC and its interfacing with Arduino.
2. To control the speed of a DC motor using PWM (Pulse Width Modulation) output from an Arduino pin.
3. To control the rotational direction (forward/reverse) of a DC motor by toggling the logic inputs of the L293D.
4. To observe and analyze the relationship between PWM duty cycle and motor speed.
5. To implement a complete motor control system integrating a potentiometer for speed adjustment and push buttons for direction control.

Theory:

An electromechanical conversion device widely used in control systems, the DC motor is designed to transform electrical energy into rotational mechanical motion. It operates on the principle of electromagnetic interaction between the armature winding and the magnetic field, producing continuous torque that results in shaft rotation. The direction and speed of rotation can be controlled by varying the applied voltage as well as reversing the polarity across the motor terminals. In addition to basic operation, DC motors exhibit important characteristics such as starting torque, speed regulation capability, efficiency, and load-dependent speed variation. They are also known for having a nearly linear relationship between applied voltage and rotational speed under constant load conditions.

In embedded and microcontroller-based applications, direct interfacing of a DC motor with an Arduino Uno is not feasible because the digital I/O pins are limited to a current rating of approximately 40 mA. This current is significantly lower than the requirement of typical DC motors, which often demand hundreds of milliamps or more during normal operation and even higher current during starting conditions. Moreover, motors can generate back electromotive force (back-EMF) during sudden stopping or direction changes, which may damage the microcontroller if not properly protected.

To overcome these limitations, a motor driver IC such as the L293D is used as an interface between the microcontroller and the motor. The L293D is a dual H-bridge driver that allows bidirectional control of two DC motors. It enables both direction and speed control by using logic-level input signals from the Arduino while handling higher current required by the motor

through an external power supply. The IC also provides built-in protection diodes to suppress voltage spikes generated by inductive loads.

The L293D operates on the H-bridge configuration, which consists of four switching elements arranged in a specific topology that allows current to flow in either direction through the motor. By controlling the switching states, forward and reverse rotation can be achieved efficiently. The device has 16 pins, including logic supply (Vcc1), motor supply (Vcc2), enable pins, input control pins, and output pins connected to the motor terminals.

Speed control is implemented using Pulse Width Modulation (PWM) signals generated by the Arduino. By varying the duty cycle of the PWM waveform, the average voltage applied to the motor is controlled, thereby adjusting its speed. Direction control is achieved by applying appropriate logic levels to the input pins, where different combinations of HIGH and LOW signals determine forward, reverse, or braking conditions.

In practical applications, this type of motor control system is widely used in robotics, automated systems, and smart home devices. It provides a simple yet effective method for achieving precise control of motion-based components. The integration of microcontrollers with motor driver ICs enhances system reliability, improves safety, and allows scalable design for multiple motor-driven operations within a single embedded platform.

Required Apparatus:

Table 7.1: Table for Required Apparatus

SL. No.	Components	Specification	Quantity
1.	Arduino Uno R3	ATmega328P, 5V, 16 MHz	1
2.	L293D Motor Driver IC	Quadruple Half-H Driver, 600 mA per channel	1
3.	DC Motor	5V–12V, 200–300 RPM	1
4.	Potentiometer	10 kΩ	1
5.	Push Button Switch	Tactile, 4-pin	2
6.	Breadboard	Full-size 830 tie-points	1
7.	Jumper Wires	Male-to-Male, 20 cm	As required
8.	9V Battery / DC Power Supply	9V, 500 mA (for motor)	1
9.	USB Cable	Type-A to Type-B for Arduino	1
10.	PC / Laptop	Arduino IDE installed	1

Circuit Diagram:

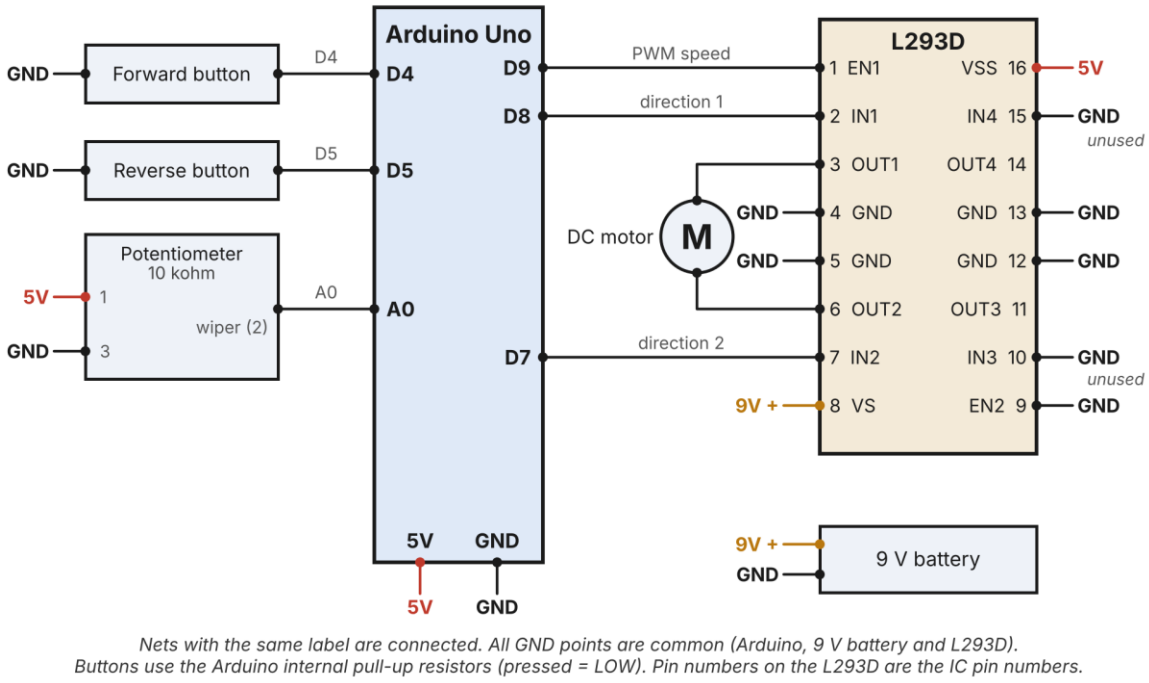


Fig. 7.1: Circuit Diagram Arduino UNO interfaced with L293D Motor Driver IC for DC Motor Control

Experimental Setup:

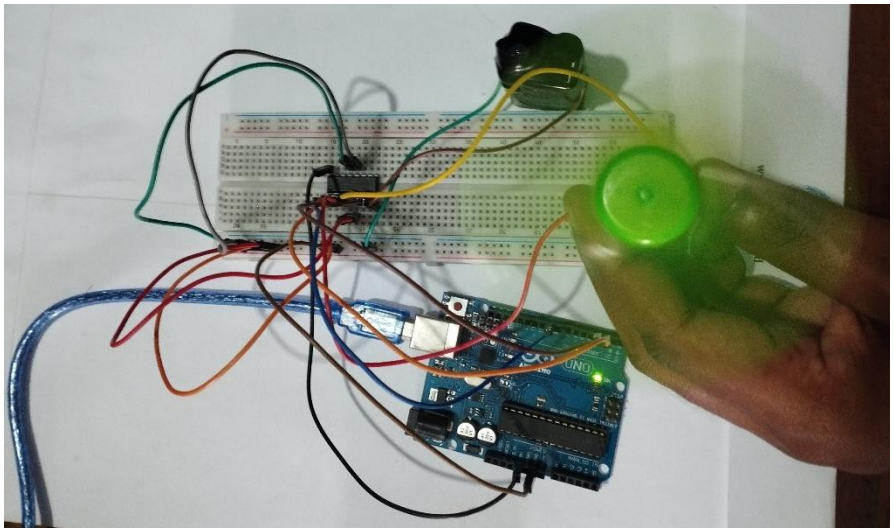


Fig. 7.2: Experimental setup showing the physical arrangement of Arduino, L293D, DC motor, and control components on breadboard

Arduino Program:

```
// DC Motor Control using Arduino and L293D
// Experiment 07 - EEE 3100 Lab

const int enA  = 9;   // PWM pin for Enable A (speed control)
const int in1  = 8;   // Direction input 1
const int in2  = 7;   // Direction input 2
const int pot  = A0;  // Potentiometer for speed control
const int btn1 = 4;   // Button: Forward
const int btn2 = 5;   // Button: Reverse

void setup() {
  pinMode(enA, OUTPUT);
  pinMode(in1, OUTPUT);
  pinMode(in2, OUTPUT);
  pinMode(btn1, INPUT_PULLUP);
  pinMode(btn2, INPUT_PULLUP);
  Serial.begin(9600);
}

void loop() {
  int potVal  = analogRead(pot);
  int speed   = map(potVal, 0, 1023, 0, 255);
  bool fwd    = (digitalRead(btn1) == LOW);
  bool rev    = (digitalRead(btn2) == LOW);

  if (fwd) {
    // Rotate Forward
    digitalWrite(in1, HIGH);
    digitalWrite(in2, LOW);
    analogWrite(enA, speed);
    Serial.print("Forward | Speed (PWM): ");
    Serial.println(speed);
  } else if (rev) {
    // Rotate Reverse
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    analogWrite(enA, speed);
    Serial.print("Reverse | Speed (PWM): ");
    Serial.println(speed);
  } else {
    // Stop
    digitalWrite(in1, LOW);
    digitalWrite(in2, LOW);
    analogWrite(enA, 0);
    Serial.println("Motor Stopped");
  }
  delay(100);
}
```

Experimental Procedure:

1. Assemble the circuit on a breadboard as per the circuit diagram (Fig. 7.1). Connect the L293D IC, Arduino, DC motor, potentiometer, and push buttons as described.

2. Verify all connections carefully, paying attention to the Enable pin (pin 9 to L293D EN1), direction control pins (IN1, IN2), and the separate 9V motor supply at Vcc2 of the L293D.
3. Open the Arduino IDE on the PC, create a new sketch, and type the provided Arduino program. Verify the code for compilation errors.
4. Upload the compiled sketch to the Arduino Uno via USB cable.
5. Power on the circuit. Observe that the motor is initially stopped (neither button pressed).
6. Press Button 1 (Forward): observe the DC motor rotating in the forward direction. Slowly rotate the potentiometer and note how the motor speed increases or decreases.
7. Release Button 1 and press Button 2 (Reverse): observe the motor changing its rotational direction. Repeat the speed variation using the potentiometer.
8. Open the Arduino Serial Monitor (baud rate 9600) and observe the printed PWM values (0–255) corresponding to potentiometer positions.
9. Record the observed PWM duty cycle values alongside the corresponding qualitative motor speeds (stopped, slow, medium, fast) in the results section.
10. Release all buttons to confirm the motor stops and the PWM output drops to 0.

Results and Discussion:

The experiment successfully demonstrated bidirectional speed control of a DC motor using an Arduino and the L293D motor driver IC. It was observed that when Button 1 (Forward) was pressed, the inputs were set to IN1 = HIGH and IN2 = LOW, resulting in clockwise rotation of the motor. When Button 2 (Reverse) was pressed, the inputs changed to IN1 = LOW and IN2 = HIGH, which smoothly reversed the motor direction.

Speed control was achieved by varying the potentiometer, which produced an analog input range from 0 to 1023. This value was mapped to a PWM output range of 0 to 255 and applied to the enable pin using analogWrite(), allowing the motor speed to increase proportionally with the duty cycle.

From the Serial Monitor readings, it was noted that at minimum potentiometer position, the PWM value was approximately 0, causing the motor to stop. At the midpoint, the PWM value was around 128, corresponding to nearly 50% duty cycle and medium speed. At maximum position, the PWM reached 255, resulting in full motor speed.

The L293D's internal H-Bridge effectively supplied the required current for the DC motor, which cannot be directly driven by the Arduino. Its built-in flyback diodes protected the circuit from voltage spikes generated by the inductive load. Overall, the experiment confirms that PWM-based duty cycle control is an efficient method for achieving analog-like speed regulation in digital embedded systems.

Conclusion:

This experiment successfully demonstrated DC motor control using an Arduino Uno and the L293D H-Bridge motor driver IC. The L293D effectively interfaced low-voltage Arduino signals with the higher current required by the motor. PWM control using analogWrite() enabled smooth speed variation, while switching the IN1 and IN2 pins allowed forward and reverse rotation. The potentiometer provided real-time speed adjustment, and the push-button confirmed correct H-Bridge direction control. Overall, the experiment reinforced key concepts of embedded motor control systems with applications in robotics, automation, and mechatronics.

Reference:

1. S D Robotics, "Motor Driver in depth | H-Bridge | L293D, L298N, TB6612FNG, VNH2SP30," YouTube, Jun. 13, 2021. [Online]. Available: https://www.youtube.com/watch?v=PVyAcgYkzDs
2. Engineers & Electronics, "L293D Motor Driver with Arduino | full Explanation | (Arduino Series - Part 09)," YouTube, Dec. 28, 2018. [Online]. Available: https://www.youtube.com/watch?v=zhWGeJnIfeY
3. Texas Instruments, "L293D Quadruple Half-H Drivers Datasheet," SLRS008B, Texas Instruments Inc. [Online]. Available: https://www.ti.com/lit/ds/symlink/l293d.pdf
4. Arduino, "analogWrite() Reference," Arduino Documentation. [Online]. Available: https://www.arduino.cc/reference/en/language/functions/analog-io/analogwrite/